МЕТОДИЧЕСКИЕ УКАЗАНИЯ
к лабораторной работе № 4
Тема: «Алгоритмическая обработка числовых рядов: формирование массивов, расчёт статистических показателей (среднее, медиана, размах) и скользящей средней»

Вариант А — базовая траектория · Вариант Б — продвинутая траектория

Параметр	Значение
Дисциплина	«Программирование и прототипирование ИТ-решений»
Направление подготовки	09.03.03 Прикладная информатика
Профиль	Бизнес и системная аналитика
Семестр / трудоёмкость	1 семестр, 4 академических часа (2 пары)
Среда разработки	Visual Studio Code
Форма контроля	отчёт о выполнении работы + устная защита
Обеспеченность ПО	Python 3.11+, Git, VS Code (подготовлены в ЛР № 1)

1. Цель работы

Научиться программно формировать и обрабатывать массивы (списки) числовых данных, вычислять базовые статистические показатели (среднее арифметическое, медиану, минимум, максимум, размах вариации) без использования сторонних библиотек и реализовывать алгоритм сглаживания временного ряда методом простой скользящей средней (Simple Moving Average, SMA) на примере суточных замеров дебита нефтяной скважины.

В результате выполнения работы студент научится:

- формировать массивы числовых данных (list[float]) в Python, обращаться к элементам по индексу и выделять подмассивы с помощью срезов (slices);
- реализовывать расчёт статистических характеристик ряда (среднее, медиана для чётного и нечётного числа элементов, размах вариации);
- программировать алгоритм скользящей средней (SMA) с произвольным окном сглаживания k с использованием циклов и срезов rates[i : i + k];
- анализировать динамику технологического ряда (тренд роста/падения дебита) и выводить наглядную консольную таблицу и текстовый график;
- сверять машинный расчёт медианы и скользящей средней с ручным контрольным пересчётом и фиксировать историю работы в Git.

2. Обязательное ядро (одинаково для обоих вариантов)

- скрипт формирует массив индивидуальных замеров дебита по номеру студента N (плюс поддерживает ручной ввод или добавление замеров);

- статистические показатели (среднее, медиана, размах) и скользящая средняя вычисляются собственным алгоритмом на чистом Python (без импорта statistics, numpy, pandas);
- алгоритм расчёта медианы корректно работает как для нечётного, так и для чётного количества элементов и не портит порядок исходного массива (используется sorted());
- реализована функция вычисления скользящей средней moving_average(series, window) с проверкой допустимости размера окна;
- результаты выводятся в виде выровненной таблицы f-строками с точностью до двух знаков после запятой (:.2f);
- в репозитории Git сделано не менее трёх осмысленных коммитов (инициализация → расчётные алгоритмы → документация);
- в отчёт включён ручной контрольный пересчёт медианы, размаха и первых двух точек скользящей средней для ВАШИХ индивидуальных данных;
- выполнены ВСЕ 7 обязательных упражнений на понимание (раздел 8) с фиксацией результатов и скриншотов в отчёте;
- README.md содержит назначение, формулы расчёта, пример запуска и пример табличного вывода.

3. Глоссарий: все термины работы

Перед началом работы прочитайте термины. К ним можно возвращаться по ходу выполнения шагов.

3.1. Списки (массивы), индексация и срезы в Python

Термин	Что это значит
Список (list)	Упорядоченная изменяемая коллекция элементов: rates = [52.0, 54.5, 51.0]. В других языках часто называется массивом.
Индекс (index)	Номер позиции элемента в списке, начиная с нуля: rates[0] — первый элемент, rates[-1] — последний элемент.
Длина списка len(s)	Количество элементов в списке: len([10, 20, 30]) вернёт 3.
Срез (slice) s[a:b]	Подсписок из элементов от индекса a (включительно) до индекса b (не включая b). Длина среза s[i : i + k] равна ровно k.
sorted(s)	Встроенная функция, которая возвращает НОВЫЙ отсортированный по возрастанию список, не изменяя исходный список s.
s.sort()	Метод, который сортирует сам список s «на месте» (in-place) и возвращает None. Для временных рядов опасен тем, что разрушает хронологию замеров!
Списковое включение	Компактный способ создания списка: [round(x, 2) for x in series].

3.2. Статистические показатели числового ряда

Термин	Что это значит
Временной ряд	Последовательность замеров показателя, упорядоченная по времени (например, суточный дебит скважины с 1-го по 10-й день).

Среднее (Mean)	Сумма всех элементов ряда, делённая на их количество: mean = sum(s) / len(s). Чувствительно к резким выбросам.
Медиана (Median)	Серединное значение отсортированного ряда. Если длина n нечётная — элемент с индексом n // 2; если чётная — полусумма двух центральных элементов: (b[n // 2 - 1] + b[n // 2]) / 2. Устойчива к выбросам!
Размах вариации (Range)	Разность между максимальным и минимальным значениями ряда: R = max(s) - min(s). Показывает амплитуду колебаний дебита.
Выброс (Аномалия)	Резкое кратковременное отклонение замера от общего уровня (например, при продувке скважины или сбое расходомера).

3.3. Алгоритм скользящей средней (Moving Average)

Термин	Что это значит
Скользящая средняя (SMA)	Алгоритм сглаживания ряда, при котором в каждой позиции вычисляется среднее арифметическое из k соседних замеров.
Окно сглаживания (k)	Количество подряд идущих точек, попадающих в одно усреднение (например, при k = 3 усредняются замеры за 3 смежных дня).
Сдвиг окна	На каждом шаге цикла окно [i : i + k] сдвигается вправо на 1 индекс. Из ряда длины n при окне k получается ровно n - k + 1 сглаженных значений.
enumerate(s, start=1)	Функция для одновременного перебора номера шага (дня) и самого значения в цикле for day, val in enumerate(rates, start=1):.

4. Необходимое программное обеспечение

Всё окружение подготовлено в лабораторной работе № 1. Убедитесь, что оно на месте:

Инструмент	Назначение	Проверка
Python 3.11+	интерпретатор языка	python --version
Visual Studio Code	редактор кода и терминал	окно VS Code открывается
Git	система контроля версий	git --version
Виртуальное окружение	изоляция зависимостей проекта	(.venv) в начале строки терминала

5. Общая подготовка (после ЛР № 1–3)

Шаг 5.1. Проверить инструменты

```
python --version
git --version
```

Зачем это делаем: Записываем обе версии в отчёт — это подтверждает готовность рабочего окружения.

Шаг 5.2. Создать папку проекта

Создайте отдельную папку под лабораторную работу № 4 (имя — по выбранному варианту, см. разделы 6 и 7):

```
mkdir series-basic
```

Зачем это делаем: Отдельная папка = отдельный чистый Git-репозиторий для работы со списками и алгоритмами.

Шаг 5.3. Создать и активировать виртуальное окружение

```
python -m venv .venv
.\.venv\Scripts\Activate
```

Выберите интерпретатор в VS Code: Ctrl + Shift + P → Python: Select Interpreter → .venv.

6. Вариант А (базовый): «Статистический анализ и сглаживание ряда дебитов скважины»

6.1. Что мы делаем

Создаём консольный скрипт, который формирует числовой ряд суточных дебитов скважины за 10 суток, вычисляет базовые статистические показатели (среднее, медиану, минимум, максимум и размах вариации) и строит сглаженный ряд методом простой скользящей средней с окном k = 3.

Это классическая задача аналитика нефтепромысла: реальные замеры дебита всегда «шумят» (колеблются изо дня в день), а скользящая средняя убирает случайный шум и показывает истинный тренд работы пласта.

6.2. Индивидуальные исходные данные (Вариант А)

Чтобы у каждого студента был свой уникальный ряд из 10 замеров, базовый уровень дебита B и поправки вычисляются из вашего номера N по журналу группы (или двух последних цифр зачётки):

Формула генерации индивидуального ряда из 10 замеров (Вариант А):

```
N = ваш номер по журналу (например, N = 5)
B = 40.0 + 2.0 * N          # базовый дебит скважины, м³/сут (для N = 5: B = 50.0)
S = (N % 5) + 1.0           # шаг колебаний, м³/сут (для N = 5: S = 1.0)

# Формула 10 суточных замеров дебита Q (индексы от 0 до 9, дни с 1-го по 10-й):
День 1: B + 0.0             День 6:  B + 1.0 * S
День 2: B + 2.0 * S         День 7:  B + 4.0 * S
День 3: B - 1.0 * S         День 8:  B + 2.0 * S
День 4: B + 3.0 * S         День 9:  B - 2.0 * S
```

```
День 5: B + 0.0             День 10: B + 5.0 * S
```

Перед запуском программы выпишите на листе бумаги все 10 чисел вашего ряда для своего N. Например, при N = 5 (B = 50.0, S = 1.0) ряд будет таким: [50.0, 52.0, 49.0, 53.0, 50.0, 51.0, 54.0, 52.0, 48.0, 55.0].

Шаг 1. Создать проект

```
mkdir series-basic
cd series-basic
python -m venv .venv
.\.venv\Scripts\Activate
```

Создайте в корне папки файл main.py (вариант А — однофайловый скрипт).

Шаг 2. Написать код

Файл main.py — изучите каждую функцию и скопируйте этот каркас:

Полный код файла main.py (Вариант А):

```
"""Алгоритмическая обработка числового ряда дебитов скважины (вариант А)."""

WINDOW_SIZE = 3  # ширина окна скользящей средней (суток)


def generate_rate_series(student_n: int) -> list[float]:
    """Формирует индивидуальный ряд из 10 суточных замеров дебита по номеру N."""
    base = 40.0 + 2.0 * student_n
    step = float((student_n % 5) + 1)
    offsets = [0.0, 2.0, -1.0, 3.0, 0.0, 1.0, 4.0, 2.0, -2.0, 5.0]
    series: list[float] = []
    for k in offsets:
        series.append(round(base + k * step, 2))
    return series


def calc_mean(series: list[float]) -> float:
    """Вычисляет среднее арифметическое числового ряда."""
    if len(series) == 0:
        raise ValueError("Нельзя вычислить среднее для пустого списка")
    total = 0.0
    for val in series:
        total += val
    return total / len(series)


def calc_median(series: list[float]) -> float:
    """Вычисляет медиану ряда (работает для чётной и нечётной длины)."""
    n = len(series)
    if n == 0:
        raise ValueError("Нельзя вычислить медиану для пустого списка")
    ordered = sorted(series)  # создаём отсортированную копию, не ломая хронологию!
    mid = n // 2
    if n % 2 == 1:
        return ordered[mid]
```

```
    else:
        return (ordered[mid - 1] + ordered[mid]) / 2.0


def calc_range(series: list[float]) -> tuple[float, float, float]:
    """Возвращает кортеж (минимум, максимум, размах вариации) без встроенных min/max."""
    if len(series) == 0:
        raise ValueError("Пустой список")
    min_val = series[0]
    max_val = series[0]
    for val in series[1:]:
        if val < min_val:
            min_val = val
        if val > max_val:
            max_val = val
    return min_val, max_val, max_val - min_val


def calc_moving_average(series: list[float], window: int) -> list[float]:
    """Вычисляет простую скользящую среднюю (SMA) с окном window."""
    if window <= 0 or window > len(series):
        raise ValueError(f"Некорректный размер окна: {window}")
    sma_values: list[float] = []
    # Количество положений окна равно len(series) - window + 1
    for i in range(len(series) - window + 1):
        window_slice = series[i : i + window]
        avg = calc_mean(window_slice)
        sma_values.append(round(avg, 2))
    return sma_values


def main() -> None:
    print("АЛГОРИТМИЧЕСКАЯ ОБРАБОТКА РЯДА ДЕБИТОВ СКВАЖИНЫ")
    print("=" * 54)
    raw_n = input("Введите ваш номер по журналу N: ").strip()
    student_n = int(raw_n) if raw_n.isdigit() and int(raw_n) > 0 else 5

    rates = generate_rate_series(student_n)
    mean_val = calc_mean(rates)
    median_val = calc_median(rates)
    min_v, max_v, range_v = calc_range(rates)
    sma_list = calc_moving_average(rates, WINDOW_SIZE)

    print(f"\nИсходный ряд (10 суток): {rates}")
    print(f"Отсортированный ряд:     {sorted(rates)}")

    print("\nСТАТИСТИЧЕСКИЕ ПОКАЗАТЕЛИ РЯДА")
    print("-" * 54)
    print(f"Среднее арифметическое:  {mean_val:8.2f} м³/сут")
    print(f"Медиана ряда:            {median_val:8.2f} м³/сут")
    print(f"Минимум / Максимум:      {min_v:8.2f} / {max_v:.2f} м³/сут")
    print(f"Размах вариации R:       {range_v:8.2f} м³/сут")

    print(f"\nТАБЛИЦА СГЛАЖИВАНИЯ (Скользящая средняя SMA, окно k = {WINDOW_SIZE})")
    print("-" * 54)
    print(f"{'Сутки':<8}{'Факт Q, м³/сут':>16}{'Окно среза':>16}{'SMA(3), м³/сут':>14}")
    print("-" * 54)
    for idx, q_val in enumerate(rates):
        day = idx + 1
        if day < WINDOW_SIZE:
```

```
            print(f"{day:<8}{q_val:>16.2f}{'—':>16}{'— (накопление)':>14}")
        else:
            sma_idx = idx - WINDOW_SIZE + 1
            win_str = f"[{day - 2}..{day}]"
            print(f"{day:<8}{q_val:>16.2f}{win_str:>16}{sma_list[sma_idx]:>14.2f}")


if __name__ == "__main__":
    main()
```

Зачем это делаем: Обратите внимание на две ключевые детали: 1) в calc_median мы используем sorted(series), а не series.sort(), чтобы хронологический порядок по суткам в списке rates не разрушился перед расчётом скользящей средней; 2) в calc_range мы написали классический линейный поиск экстремумов в один проход цикла.

Шаг 3. Запустить и проверить

```
python main.py
```

Введите свой номер N. Пример вывода программы для N = 5:

Пример вывода программы (Вариант А, N = 5):

```
АЛГОРИТМИЧЕСКАЯ ОБРАБОТКА РЯДА ДЕБИТОВ СКВАЖИНЫ
==================================================
Введите ваш номер по журналу N: 5

Исходный ряд (10 суток): [50.0, 52.0, 49.0, 53.0, 50.0, 51.0, 54.0, 52.0, 48.0, 55.0]
Отсортированный ряд:     [48.0, 49.0, 50.0, 50.0, 51.0, 52.0, 52.0, 53.0, 54.0, 55.0]

СТАТИСТИЧЕСКИЕ ПОКАЗАТЕЛИ РЯДА
--------------------------------------------------
Среднее арифметическое:     51.40 м³/сут
Медиана ряда:               51.50 м³/сут
Минимум / Максимум:         48.00 / 55.00 м³/сут
Размах вариации R:           7.00 м³/сут

ТАБЛИЦА СГЛАЖИВАНИЯ (Скользящая средняя SMA, окно k = 3)
--------------------------------------------------------
Сутки     Факт Q, м³/сут      Окно среза  SMA(3), м³/сут
--------------------------------------------------------
1                  50.00               —  — (накопление)
2                  52.00               —  — (накопление)
3                  49.00         [1..3]          50.33
4                  53.00         [2..4]          51.33
5                  50.00         [3..5]          50.67
6                  51.00         [4..6]          51.33
7                  54.00         [5..7]          51.67
8                  52.00         [6..8]          52.33
9                  48.00         [7..9]          51.33
10                 55.00        [8..10]          51.67
```

Шаг 4. Ручной контрольный пересчёт (обязательно в отчёт)

Пересчитайте вручную на калькуляторе для своих 10 чисел медиану, размах и первые два значения скользящей средней SMA(3) и запишите цепочку в отчёт. Пример для N = 5:

Образец ручного контрольного пересчёта для отчёта:

```
1) Отсортированный ряд из 10 чисел (индексы 0..9):
   [48.0, 49.0, 50.0, 50.0, 51.0, 52.0, 52.0, 53.0, 54.0, 55.0]
2) Медиана (длина n = 10 — чётная, берём элементы с индексами 4 и 5):
   Me = (ordered[4] + ordered[5]) / 2 = (51.0 + 52.0) / 2 = 51.50 м³/сут
3) Размах вариации:
   R = max - min = 55.0 - 48.0 = 7.00 м³/сут
4) Первые два значения скользящей средней (окно k = 3):
   SMA_1 (дни 1..3) = (50.0 + 52.0 + 49.0) / 3 = 151.0 / 3 = 50.333... ≈ 50.33 м³/сут
   SMA_2 (дни 2..4) = (52.0 + 49.0 + 53.0) / 3 = 154.0 / 3 = 51.333... ≈ 51.33 м³/сут
```

Зачем это делаем: Ручной пересчёт чётной медианы и первых шагов скользящего окна — главное доказательство на защите, что вы понимаете механику сдвига среза series[i : i + k].

Шаг 5. Закоммитить в Git

```
git init
git add main.py
git commit -m "Вариант А: расчёт статистик ряда и скользящей средней"
```

Создайте файл .gitignore (исключите .venv/ и __pycache__/) и файл README.md (с описанием проекта и примером запуска), затем выполните ещё два коммита:

```
git add .gitignore
git commit -m "Добавить .gitignore"
git add README.md
git commit -m "Добавить документацию README с примером расчёта"
```

7. Вариант Б (продвинутый): «Аналитическая станция мониторинга временных рядов скважины»

7.1. Что мы делаем

Разрабатываем модульный пакет с разделением конфигурации, математического ядра и консольного интерфейса. Программа анализирует 12-суточный ряд дебитов, сравнивает сглаживание двумя разными окнами (короткое окно k1 = 3 и длинное окно k2 = 5), вычисляет отклонения фактического дебита от сглаженного тренда, определяет направление тренда и строит в консоли текстовую гистограмму.

7.2. Индивидуальные исходные данные (Вариант Б)

В варианте Б генерируется ряд из 12 суточных замеров, включающий один выраженный пиковый скачок на 8-е сутки (чтобы наглядно увидеть, как медиана и среднее по-разному реагируют на выбросы):

Параметры индивидуального ряда из 12 суток (Вариант Б):

```
N = ваш номер по журналу (например, N = 5)
B = 55.0 + 2.5 * N               # базовый уровень дебита, м³/сут (при N = 5: B = 67.5)
S = (N % 4) + 1.5                # амплитуда суточных колебаний, м³/сут (при N = 5: S = 2.5)
Коэффициенты на 12 суток: [0, 1, -1, 2, 0, -2, 1, 8, -1, 2, 1, 3]
(На 8-е сутки коэффициент равен +8*S — имитация кратковременного технологического всплеска).
```

Шаг 1. Создать проект со структурой пакета (src-layout)

```
mkdir series-advanced
cd series-advanced
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

Создайте структуру каталогов и файлов проекта:

Структура проекта Варианта Б:

```
series-advanced/
├── .gitignore
├── README.md
└── src/
    └── well_series/
        ├── __init__.py
        ├── config.py
        ├── analytics.py
        └── main.py
```

Шаг 2. Написать код модулей

Файл src/well_series/__init__.py:

```
"""Пакет алгоритмической обработки временных рядов скважины."""

__version__ = "0.4.0"
```

Файл src/well_series/config.py — настройки окон сглаживания и генерации:

Файл src/well_series/config.py:

```
"""Конфигурационные константы анализа временных рядов."""

SHORT_WINDOW = 3          # короткое окно скользящей средней (суток)
LONG_WINDOW = 5           # длинное окно скользящей средней (суток)
BAR_MAX_WIDTH = 24        # максимальная ширина текстового графика в символах
OFFSETS_12D = [0.0, 1.0, -1.0, 2.0, 0.0, -2.0, 1.0, 8.0, -1.0, 2.0, 1.0, 3.0]
```

Файл src/well_series/analytics.py — чистые функции расчёта статистик и скользящих средних:

Файл src/well_series/analytics.py:

```
"""Математическое ядро: статистики ряда, скользящая средняя и оценка тренда."""

from well_series import config


def build_series_12d(student_n: int) -> list[float]:
    """Генерирует 12-суточный ряд дебитов для студента с номером N."""
    base = 55.0 + 2.5 * student_n
    step = float(student_n % 4) + 1.5
    return [round(base + k * step, 2) for k in config.OFFSETS_12D]


def calc_mean(series: list[float]) -> float:
    """Среднее арифметическое числового ряда."""
    if not series:
        raise ValueError("Пустой числовой ряд")
    total = 0.0
    for x in series:
        total += x
    return total / len(series)


def calc_median(series: list[float]) -> float:
    """Медиана числового ряда (без изменения исходного списка)."""
    n = len(series)
    if n == 0:
        raise ValueError("Пустой числовой ряд")
    ordered = sorted(series)
    mid = n // 2
    if n % 2 == 1:
        return ordered[mid]
    return (ordered[mid - 1] + ordered[mid]) / 2.0


def calc_extremes_and_range(series: list[float]) -> tuple[float, float, float]:
    """Возвращает (min_val, max_val, range_val) за один проход цикла."""
    if not series:
        raise ValueError("Пустой числовой ряд")
    lo = hi = series[0]
    for x in series[1:]:
        if x < lo:
            lo = x
        if x > hi:
            hi = x
    return lo, hi, round(hi - lo, 2)


def calc_sma_aligned(series: list[float], window: int) -> list[float | None]:
    """Возвращает список длины len(series), где первые (window - 1) элементов равны None,
    а остальные содержат скользящую среднюю SMA(window)."""
    if window <= 0 or window > len(series):
        raise ValueError(f"Окно {window} недопустимо для ряда длины {len(series)}")
    result: list[float | None] = [None] * (window - 1)
    for i in range(len(series) - window + 1):
        chunk = series[i : i + window]
        result.append(round(calc_mean(chunk), 2))
```

```
    return result


def detect_trend(sma_aligned: list[float | None]) -> str:
    """Определяет общий тренд по первой и последней доступной точке скользящей средней."""
    valid = [x for x in sma_aligned if x is not None]
    if len(valid) < 2:
        return "недостаточно данных"
    diff = valid[-1] - valid[0]
    if diff > 1.0:
        return f"восходящий (+{diff:.2f} м³/сут)"
    elif diff < -1.0:
        return f"нисходящий ({diff:.2f} м³/сут)"
    return f"стабильный ({diff:+.2f} м³/сут)"
```

Файл src/well_series/main.py — точка входа и построение сводной таблицы:

Файл src/well_series/main.py:

```
"""Консольный интерфейс аналитической станции временных рядов (вариант Б)."""

from well_series import analytics, config


def render_bar(val: float, min_v: float, max_v: float) -> str:
    """Строит текстовую полоску # пропорционально значению в диапазоне [min_v; max_v]."""
    span = max_v - min_v
    if span <= 0:
        return "#" * 10
    ratio = (val - min_v) / span
    count = 1 + int(round(ratio * (config.BAR_MAX_WIDTH - 1)))
    return "#" * count


def main() -> None:
    print("АНАЛИТИЧЕСКАЯ СТАНЦИЯ ОБРАБОТКИ ВРЕМЕННЫХ РЯДОВ (ВАРИАНТ Б)")
    print("=" * 72)
    raw = input("Введите ваш номер по журналу N: ").strip()
    student_n = int(raw) if raw.isdigit() and int(raw) > 0 else 5

    rates = analytics.build_series_12d(student_n)
    mean_v = analytics.calc_mean(rates)
    med_v = analytics.calc_median(rates)
    min_v, max_v, rng_v = analytics.calc_extremes_and_range(rates)

    sma3 = analytics.calc_sma_aligned(rates, config.SHORT_WINDOW)
    sma5 = analytics.calc_sma_aligned(rates, config.LONG_WINDOW)
    trend_str = analytics.detect_trend(sma3)

    print(f"\nХронологический ряд (12 сут.): {rates}")
    print("-" * 72)
    print(f"Среднее: {mean_v:.2f} | Медиана: {med_v:.2f} | Разница (Среднее - Медиана): {mean_v -
med_v:+.2f}")
    print(f"Минимум: {min_v:.2f} | Максимум: {max_v:.2f} | Размах вариации R: {rng_v:.2f}
м³/сут")
    print(f"Оценка тренда по SMA({config.SHORT_WINDOW}): {trend_str}")
```

```
    print("\nСВОДНАЯ ТАБЛИЦА СГЛАЖИВАНИЯ И ВИЗУАЛИЗАЦИИ РЯДА")
    print("-" * 72)
    print(f"{'День':<6}{'Q факт':>10}{'SMA(3)':>10}{'SMA(5)':>10}{'Откл. от SMA(3)':>16}
{'Профиль дебита'}")
    print("-" * 72)

    for idx, q in enumerate(rates):
        day = idx + 1
        s3 = sma3[idx]
        s5 = sma5[idx]
        s3_str = f"{s3:10.2f}" if s3 is not None else f"{'—':>10}"
        s5_str = f"{s5:10.2f}" if s5 is not None else f"{'—':>10}"
        dev_str = f"{q - s3:+16.2f}" if s3 is not None else f"{'—':>16}"
        bar = render_bar(q, min_v, max_v)
        print(f"{day:<6}{q:>10.2f}{s3_str}{s5_str}{dev_str}  {bar}")


if __name__ == "__main__":
    main()
```

Зачем это делаем: Функция calc_sma_aligned дополняет начало списка значениями None для первых (window - 1) суток, чтобы длина списка SMA в точности равнялась длине исходного ряда (12 элементов). Благодаря этому при выводе таблицы индексы суток, фактического дебита, SMA(3) и SMA(5) идеально совпадают!

Шаг 3. Запустить и проверить

```
$env:PYTHONPATH="src"
python -m well_series.main
```

Пример вывода программы для N = 5:

Пример вывода программы (Вариант Б, N = 5):

```
АНАЛИТИЧЕСКАЯ СТАНЦИЯ ОБРАБОТКИ ВРЕМЕННЫХ РЯДОВ (ВАРИАНТ Б)
========================================================================
Введите ваш номер по журналу N: 5

Хронологический ряд (12 сут.): [67.5, 70.0, 65.0, 72.5, 67.5, 62.5, 70.0, 87.5, 65.0, 72.5, 70.0,
75.0]
------------------------------------------------------------------------
Среднее: 70.42 | Медиана: 70.00 | Разница (Среднее - Медиана): +0.42
Минимум: 62.50 | Максимум: 87.50 | Размах вариации R: 25.00 м³/сут
Оценка тренда по SMA(3): восходящий (+5.00 м³/сут)

СВОДНАЯ ТАБЛИЦА СГЛАЖИВАНИЯ И ВИЗУАЛИЗАЦИИ РЯДА
------------------------------------------------------------------------
День      Q факт    SMA(3)    SMA(5)  Откл. от SMA(3)  Профиль дебита
------------------------------------------------------------------------
1          67.50         —         —                —  ######
2          70.00         —         —                —  ########
3          65.00     67.50         —            -2.50  ###
```

```
4          72.50     69.17         –           +3.33  ##########
5          67.50     68.33     68.50           -0.83  ######
6          62.50     67.50     67.50           -5.00  #
7          70.00     66.67     67.50           +3.33  ########
8          87.50     73.33     72.00          +14.17  ########################
9          65.00     74.17     70.50           -9.17  ###
10         72.50     75.00     71.50           -2.50  ##########
11         70.00     69.17     73.00           +0.83  ########
12         75.00     72.50     74.00           +2.50  #############
```

Шаг 4. Ручной контрольный пересчёт (Вариант Б)

Выполните в отчёте ручной пересчёт для своих 12 чисел: 1) медианы (полусумма 6-го и 7-го элементов отсортированного списка с индексами 5 и 6); 2) размаха R; 3) первой точки SMA(3) за дни 1..3 и первой точки SMA(5) за дни 1..5.

Шаг 5. Закоммитить в ветке Git

```
git init
git switch -c feature/time-series
git add src/well_series/config.py src/well_series/analytics.py
git commit -m "Реализовать расчёт статистик ряда и скользящей средней"
git add src/well_series/main.py .gitignore README.md
git commit -m "Добавить табличный вывод, график и документацию"
git switch -c main
git merge feature/time-series
```

8. Обязательные упражнения для обоих вариантов

Все семь упражнений выполняют ВСЕ студенты — и в варианте А, и в варианте Б. Скриншоты и ответы по каждому упражнению обязательно включаются в отчёт.

Упражнение 1. Нечётная vs чётная длина ряда (проверка медианы)

Временно добавьте в конец вашего списка замеров ещё одно число (например, rates.append(60.0)), чтобы длина ряда стала нечётной (11 для варианта А или 13 для варианта Б), и запустите скрипт.

Зачем это делаем: Для чётной и нечётной длины массива работают РАЗНЫЕ ветки if n % 2 == 1 в функции calc_median. Мы обязаны протестировать обе ветки!

Проверка понимания (в отчёт): Приведите в отчёте скриншот запуска с нечётной длиной и покажите вручную, какой именно центральный элемент отсортированного списка стал медианой.

Упражнение 2. Устойчивость медианы к выбросу (Аварийный всплеск)

Замените последний замер в вашем ряду на гигантский выброс — например, 500.0 м³/сут (сбой расходомера). Запустите программу и сравните, насколько сильно изменилось Среднее арифметическое и насколько изменилась Медиана.

Зачем это делаем: На реальном промысле датчики часто дают единичные сбои («шпильки»). Среднее при этом сильно искажается, а медиана почти не меняется — поэтому аналитики так ценят

медиану.

Проверка понимания (в отчёт): Запишите в отчёт таблицу «Показатель | До выброса | После выброса 500.0» для среднего, медианы и размаха и письменно объясните причину разницы.

Упражнение 3. Влияние ширины окна сглаживания (k = 2 против k = 5)

Последовательно запустите расчёт скользящей средней с маленьким окном k = 2 и с широким окном k = 5. Сравните размах колебаний (max - min) самого сглаженного ряда SMA(2) и SMA(5).

Зачем это делаем: Чем шире окно k, тем сильнее сглаживаются резкие пики, но тем больше «запаздывает» скользящая средняя и тем больше первых точек тратится на накопление окна (k - 1 точек).

Проверка понимания (в отчёт): Укажите в отчёте, сколько точек получилось в списке SMA при k = 2 и при k = 5, и проверьте формулу длины n - k + 1.

Упражнение 4. Опасность сортировки «на месте» (.sort() вместо sorted())

Проведите эксперимент: внутри функции calc_median замените строку ordered = sorted(series) на вызов метода series.sort() (и используйте series вместо ordered). Запустите программу и посмотрите, что случилось с таблицей по суткам и со значениями скользящей средней!

Зачем это делаем: В Python списки передаются в функцию по ссылке. Вызов series.sort() внутри calc_median навсегда перемешивает исходный массив замеров по возрастанию, уничтожая хронологию по дням! После снятия скриншота обязательно верните sorted(series) обратно.

Проверка понимания (в отчёт): Приложите скриншот «испорченной» хронологии и своими словами объясните разницу между sorted(s) и s.sort().

Упражнение 5. Суммарная добыча и доля пиковых суток

Добавьте в вывод расчёт суммарного объёма добытой жидкости за весь период (Q_sum) и определите, сколько процентов от общей добычи дал самый продуктивный день (max_v / Q_sum * 100).

Проверка понимания (в отчёт): Приведите код доработки, скриншот вывода и ручной проверочный расчёт доли максимальных суток.

Упражнение 6. Подсчёт числа суток выше среднего уровня (Фильтрация ряда)

Напишите функцию или списковое включение, которое отбирает номера суток, когда фактический дебит скважины был строго выше среднего арифметического по ряду (q > mean_val), и выводит их количество и список этих дней.

Проверка понимания (в отчёт): Приложите код и скриншот вывода. Всегда ли количество элементов выше среднего равно ровно половине длины ряда? Объясните почему.

Упражнение 7. Защита от вырожденных случаев (Краевые сценарии)

Проверьте поведение вашей функции calc_moving_average на трёх крайних случаях: 1) окно k = 1; 2) окно равно всей длине ряда k = len(rates); 3) окно больше длины ряда k = 20 (должно перехватываться исключение ValueError без аварийного падения программы).

Проверка понимания (в отчёт): Покажите скриншот проверки всех трёх случаев и ответьте: чему равен список SMA при k = 1 и сколько чисел остаётся в списке SMA при k = len(rates)?

9. Пример оформления отчёта

- Титульная часть: дисциплина, тема ЛР № 4, ФИО, группа, дата, вариант (А или Б), номер по журналу N;
- Цель работы и версии ПО (Python, Git, VS Code);
- Индивидуальные исходные данные: расчёт базового дебита B, шага S и выписанный массив из 10 (или 12) замеров;
- Ручной контрольный пересчёт: отсортированный ряд, медиана, размах и первые два шага скользящей средней;
- Ход работы: листинг кода и скриншот основного табличного вывода программы;
- Отчёт по всем 7 обязательным упражнениям (раздел 8) со скриншотами и пояснениями;
- Письменные ответы на контрольные вопросы (раздел 11) и общий вывод.

Пример начала отчёта (Вариант А):

```
------------------------------------------------------------
Отчёт по лабораторной работе № 4
Дисциплина: Программирование и прототипирование ИТ-решений
Студент: Иванов И.И., гр. БПИ-26-01 · Вариант А · N = 5

1. Индивидуальный ряд (B = 50.0, S = 1.0):
   Q = [50.0, 52.0, 49.0, 53.0, 50.0, 51.0, 54.0, 52.0, 48.0, 55.0]
2. Ручной контрольный пересчёт:
   - Отсортированный ряд: [48, 49, 50, 50, 51, 52, 52, 53, 54, 55]
   - Среднее = 514.0 / 10 = 51.40 м³/сут
   - Медиана (n = 10, чётная) = (51.0 + 52.0) / 2 = 51.50 м³/сут
   - Размах R = 55.0 - 48.0 = 7.00 м³/сут
   - SMA(3) за дни [1..3] = (50 + 52 + 49) / 3 = 50.33 м³/сут (совпало с программой)
------------------------------------------------------------
```

10. Частые ошибки и их решение

Симптом / Ошибка	Причина и решение
IndexError: list index out of range при расчёте SMA	Неверная верхняя граница цикла range(). Для окна k и списка длины n цикл должен идти строго до range(len(series) - window + 1).
После вызова calc_median() таблица по дням выводится по возрастанию дебита	Вы использовали метод series.sort() вместо функции sorted(series) и изменили исходный список на месте.
Медиана для 10 элементов считается неправильно (берётся только 5-й или 6-й элемент)	В чётном списке из 10 элементов (индексы 0..9) центральными являются ДВА элемента с индексами n // 2 - 1 (4) и n // 2 (5). Нужна их полусумма!
Срез series[i : i + window] берёт	Правая граница среза в Python не включается: срез series[0:3]

на 1 элемент меньше или больше	берёт элементы с индексами 0, 1 и 2 — ровно 3 штуки.
ZeroDivisionError: division by zero в функции calc_mean	В функцию передан пустой список []. Добавьте в начало проверку if len(series) == 0: raise ValueError(...).
ModuleNotFoundError: No module named 'well_series' (в варианте Б)	Забыли указать путь к папке src перед запуском: выполните $env:PYTHONPATH="src" в терминале PowerShell.

11. Контрольные вопросы для защиты

Ответьте на каждый вопрос письменно в отчёте своими словами, опираясь на написанный вами код:

- **Вопрос 1.** Чем отличается расчёт медианы для числового ряда с нечётным количеством элементов (например, 11 суток) и с чётным количеством элементов (10 или 12 суток)? Покажите строки вашего кода, которые за это отвечают.
- **Вопрос 2.** Почему внутри функции calc_median необходимо использовать функцию sorted(series), а не списочный метод series.sort()? Как именно изменится результат работы функции calc_moving_average, если массив отсортируется «на месте»?
- **Вопрос 3.** Как работает срез списка series[i : i + window] в алгоритме скользящей средней? Почему при длине ряда n = 10 и ширине окна k = 3 получается ровно 8 значений скользящей средней, а не 10?
- **Вопрос 4.** Какой статистический показатель — среднее арифметическое или медиана — сильнее реагирует на единичный аварийный выброс датчика и почему? Приведите численный пример из вашего Упражнения № 2.
- **Вопрос 5.** Как меняется поведение кривой скользящей средней при увеличении ширины окна сглаживания k (например, с k = 3 до k = 5)? В чём состоит компромисс между степенью сглаживания шума и запаздыванием оценки тренда?